

Chapter 1

AI Foundations

A practical first-principles introduction to artificial intelligence, machine learning, deep learning, reinforcement learning, and the vocabulary used by AI engineering teams.

Suggested time: 6–8 hours

Prerequisite: basic Python

Deliverable: a small, tested support-triage service and an architecture note

1. What you are building toward

An AI engineer does not merely “use a model.” They turn uncertain model behavior into a useful product: a system with data, APIs, interfaces, safety controls, evaluations, monitoring, and a cost envelope. This chapter establishes the mental model needed before discussing mathematics or individual algorithms.

Working definition. Artificial intelligence is software that performs tasks associated with perception, language, prediction, decision-making, or learning. It is an umbrella term. Machine learning, deep learning, and generative AI are important subsets—not synonyms.

Learning objectives

- Explain the difference between rules, machine learning, deep learning, and LLM applications.
- Use the terms training, inference, feature, label, parameter, loss, and evaluation correctly.
- Recognize when a deterministic program is better than AI.
- Describe why production AI needs monitoring, security, and human controls.

AI is a spectrum of approaches

Approach	What it does	Best fit	Weakness
Rules	Executes explicit logic written by people.	Known policies, calculations, eligibility.	Brittle when patterns are too complex to specify.
Classical ML	Learns a mapping from structured examples.	Churn, fraud, demand, ranking.	Needs representative labeled data.
Deep learning	Learns rich representations with neural networks.	Images, audio, complex text.	Data/compute hungry and less interpretable.
Generative AI / LLMs	Generates or transforms language and multimodal content.	Drafting, extraction, search assistance, tools.	Can hallucinate; needs guardrails and evidence.

The system view



The important idea is that a model is one component. It does not replace authorization, business logic, source-of-truth data, or accountability.

2. A short, useful history of AI

History matters because older approaches remain valuable. Most companies still use rule engines, regression, tree models, and search alongside LLMs.

Era	Core idea	What remains useful today
1950s–1970s: symbolic AI	Represent knowledge as rules and symbols.	Policy engines, formal constraints, deterministic workflows.
1980s: expert systems	Hand-built “if/then” knowledge bases.	Decision tables and compliance logic.
1990s–2000s: statistical ML	Learn patterns from data.	Classification, regression, ranking, anomaly detection.
2010s: deep learning	Large neural networks learn representations.	Vision, speech, translation, embeddings.
Late 2010s onward: transformers	Attention-based networks scale effectively on sequences.	Foundation models, code, multimodal systems, retrieval.
Current engineering focus	Models connected to tools and data.	Evaluated, secure, observable agentic workflows.

Do not confuse progress with replacement. A support workflow may use an LLM to understand a customer message, rules to decide refund eligibility, SQL to retrieve the order, and a human to approve an exception. That is usually stronger than an all-LLM system.

3. Types of AI: narrow AI, AGI, and superintelligence

Narrow AI is specialized: a recommender, fraud model, vision classifier, or assistant. All commercially deployed AI should be treated as narrow AI with bounded operating conditions. **Artificial General Intelligence (AGI)** generally means a hypothetical system able to perform a broad range of intellectual tasks at human-level adaptability. There is no universally accepted measurable threshold.

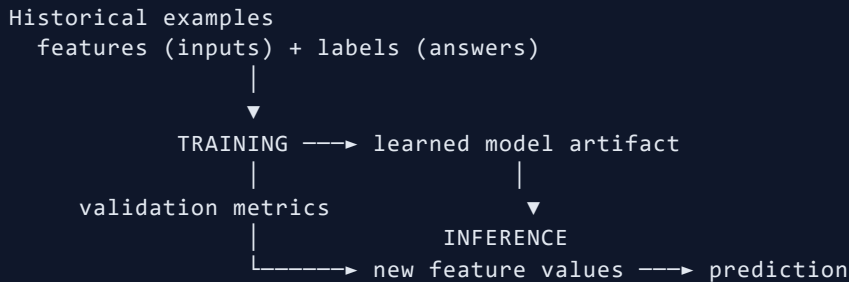
Superintelligence describes a speculative capability beyond human performance across most domains.

For an engineer, the practical question is not “Is this AGI?” It is: *What tasks, inputs, permissions, error rates, failure modes, and oversight does this system have?*

Question to ask	Engineering consequence
What decisions may it make?	Define a narrow action allowlist and approval threshold.
What evidence may it use?	Use governed sources and retrieval permissions.
What happens when uncertain?	Abstain, escalate, or choose a safe fallback.
How is correctness measured?	Create a representative evaluation set before rollout.

4. Machine learning from first principles

Machine learning learns a function from examples. A fraud model might learn $f(\text{transaction}) \rightarrow \text{probability of fraud}$. During **training**, it adjusts numerical values called **parameters** to reduce a **loss**: a number representing how wrong its predictions are. During **inference**, it applies the learned parameters to new inputs.



Classification

Predict a discrete category: fraud/not fraud, urgent/normal, approved/denied. Quality may be measured with precision, recall, F1, calibration, and business cost.

Regression

Predict a number: delivery time, demand, revenue, price. Quality may be measured with MAE, RMSE, and error by important segment.

Features, labels, and leakage

A **feature** is information available when making a prediction: order total, country, account age. A **label** is the historical answer: whether a charge was fraudulent. Data leakage occurs when training uses information that would not exist at prediction time, such as a refund field created after a fraud investigation. Leakage produces dazzling offline results and failing production models.

Minimal Python baseline

```
from sklearn.linear_model import LogisticRegression

# Features: [account_age_days, number_of_failed_payments]
X = [[400, 0], [2, 4], [700, 0], [1, 5]]
# Label: 1 means send to manual review
y = [0, 1, 0, 1]

model = LogisticRegression().fit(X, y)
review_probability = model.predict_proba([[3, 3]])[0, 1]
print(f"Review probability: {review_probability:.2f}")
```

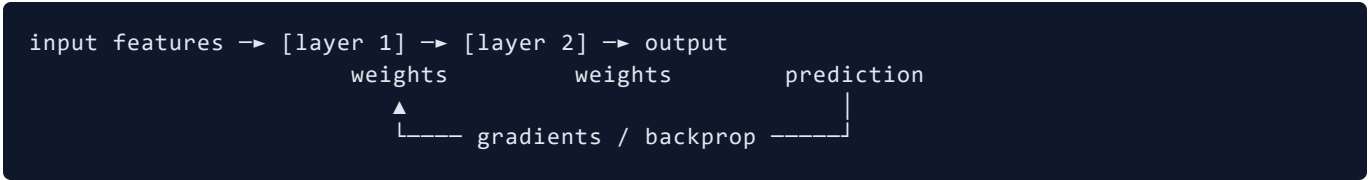
This toy model is not production-ready. Production work needs representative data, train/validation/test separation, a decision threshold, monitoring, and a procedure for human feedback.

Why a model instead of a rule?

Rules are excellent when the decision is explicit: “block an account with a verified sanctions match.” A model is useful when thousands of weak signals combine in a pattern people cannot practically enumerate. Often the right design is a rule that overrides or constrains a model.

5. Deep learning and neural networks

A neural network is a function composed of layers. Each layer applies weighted mathematical transformations; non-linear activation functions let the network represent patterns more complex than a straight line. The word “neural” is historical inspiration, not a claim that the system thinks like a brain.



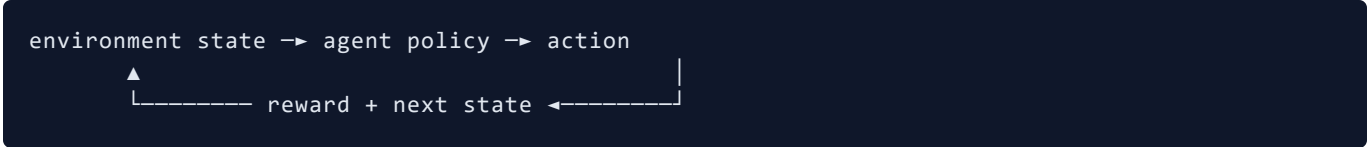
Deep learning exists because learning intermediate representations directly from raw or complex inputs can outperform hand-designed features. A vision model can learn edges, shapes, and objects. A language model can learn statistical relationships between tokens. The trade-off is more data, compute, experimentation, and more difficult explanations.

Use case	Typical model family	Why
Product images	CNN or vision transformer	Captures spatial patterns.
Speech transcription	Transformer audio model	Handles long temporal sequences.
Customer churn table	Gradient-boosted trees first	Often simpler and stronger for tabular data.
Document assistant	LLM + retrieval	Understands/generates language with source grounding.

Best practice: choose the simplest model that meets a measured requirement. A reliable logistic-regression or tree baseline is an engineering victory, not a compromise.

6. Reinforcement learning

In reinforcement learning, an **agent** observes a **state**, takes an **action**, receives a **reward**, and learns a **policy** for future actions. The challenge is delayed credit: an action can have consequences many steps later.



Industry uses include control, ranking/optimization, games, simulation, and preference optimization. It is usually a poor first choice for a business workflow: rewards are hard to design, online mistakes can be costly, and safe exploration is difficult. In LLM post-training, preference-based methods use human or

synthetic comparisons to steer responses; application engineers normally consume the resulting models rather than run RL themselves.

5. Log a redacted trace: model version, route, confidence, latency, outcome, and human correction.

Production-oriented code

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Ticket:
    text: str
    customer_id: str

def route(ticket: Ticket) -> str:
    text = ticket.text.lower()
    # Policy overrides model behavior.
    if any(term in text for term in ("stolen card", "account hacked")):
        return "security_human_review"
    if "refund" in text:
        return "refund_policy_workflow"
    return "shipping_or_general_classifier"

assert route(Ticket("My card was stolen", "cust_7")) == "security_human_review"
```

Cost note: rules are effectively free per decision; a compact ML classifier usually costs CPU milliseconds; an LLM adds variable token cost and latency. Place the expensive component only where its quality advantage changes the outcome.

9. Common mistakes and practical checks

Mistake	Better practice
Calling every automation “AI.”	State the exact task, input, decision, metric, and owner.
Choosing an LLM before defining the problem.	Build a rule/baseline and measure the improvement needed.
Using accuracy as the only metric.	Include error costs, precision/recall, latency, safety, and user outcomes.
Testing on the training data.	Use separate validation/test data; evaluate production samples.
Assuming a fluent answer is correct.	Use source evidence, structured validation, and escalation paths.
Logging all prompts and PII by default.	Minimize/redact logs and enforce retention/access controls.

10. Interview practice

1. Explain AI, ML, deep learning, and generative AI to a product manager.
2. When would you use a rule instead of a learned model?
3. Define overfitting and give two production defenses.
4. What is the difference between training and inference cost?
5. Why is a model with 99% accuracy potentially unsafe?
6. Describe a system where an LLM should not make the final decision.

7. How would you detect that a support classifier is degrading?
8. What is reward design, and why is it difficult in RL?

Chapter summary. AI engineering begins by selecting the smallest dependable approach for a defined task. Rules encode known policy. ML learns from structured examples. Deep learning learns complex representations. RL optimizes sequential reward but is difficult to apply safely. LLMs are powerful generative components, not decision makers or security boundaries. Every production system needs a measurable objective, representative evaluation, controlled actions, and monitoring.

Hands-on project: support-triage baseline

Build a small FastAPI service with `POST /triage`. Accept ticket text and a customer ID; apply policy overrides; return a route; write unit tests for the security/refund/general paths. Add a README that records the intended metric, an example failure, and what data you would collect before training a classifier. Do not use real customer data.

Further reading

- [Google: Rules of Machine Learning](#)
- [scikit-learn User Guide](#)
- [Deep Learning Book](#) (reference chapters as needed)
- [NIST AI Risk Management Framework](#)

Next chapter: Practical Mathematics for AI Engineers — vectors, probability, statistics, derivatives, gradient descent, and optimization through working examples.